Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 10a - Modern Model-Based and Offline Reinforcement Learning

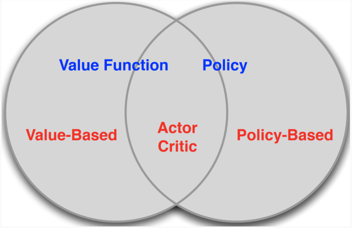
Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Modern Model-based RL

RL Paradigms: From Clean Categories to Hybrids

We outlined characteristics of different RL methods and tried to create very clear distinctions between them.



In practice, there are various approaches that combine ideas from different paradigms.

Which approaches have we seen?

From Classical to Modern Model-Based RL

Classical model-based RL:

- Learn a model of the environment.
- Plan using the learned model.
- Execute actions in the real environment.

Modern model-based RL keeps the same idea, but changes the model:

- Models are learned with neural networks.
- Planning is often performed over short horizons.
- Uncertainty is explicitly represented.
- Models may operate in latent space rather than directly in state space.

Why Did Model-Based RL Become Popular Again?

Model-based RL was traditionally difficult because:

- learned models were inaccurate,
- small model errors accumulated over long rollouts,
- high-dimensional observations were hard to model,
- planning with learned models was computationally expensive.

Why Did Model-Based RL Become Popular Again?

Model-based RL was traditionally difficult because:

- learned models were inaccurate,
- small model errors accumulated over long rollouts,
- high-dimensional observations were hard to model,
- planning with learned models was computationally expensive.

Recent progress changed this:

- deep learning improved dynamics models,
- GPUs made simulation and planning faster,
- ensembles improved uncertainty estimation,
- latent-space models reduced the prediction problem.

Challenges

Model-based RL separates learning into the following parts:

1. collect experience in the real environment,
2. learn a predictive model of dynamics and possibly rewards,
3. use the model to plan or improve a policy.

A common model is

$$\hat{p}_\eta(s' \mid s, a), \qquad \hat{r}_\eta(s, a).$$

In small MDPs, a model can be a table:

$$(s, a) \mapsto \hat{P}(\cdot \mid s, a), \hat{R}(s, a).$$

In large or continuous domains, we use function approximators:

$$(s, a) \mapsto f_\eta(s, a)$$

or

$$(s, a) \mapsto p_\eta(s', r \mid s, a).$$

Examples include:

- neural dynamics models,
- probabilistic ensembles,
- latent world models.

Model-Based RL Design Dilemmas

Statistics	Models	Control
How much data is needed to learn a useful model?	What model class should represent dynamics, rewards, and uncertainty?	How do we plan or improve a policy using an imperfect model?

- Accurate models may be computationally expensive.
- Planning in high-dimensional observations can be difficult.
- Sometimes a simpler model is better for control.

Representing the Model

What Should the Model Predict?

A learned model may predict:

- next low-dimensional state,
- next image observation,
- reward,
- termination probability,
- latent state instead of raw observations.

The best model for prediction is not always the best model for control. The model should preserve information relevant for decisions.

A world model is a learned predictive model of the environment.

It can answer questions such as:

- What will happen if I take this action?
- What states are reachable?
- Which actions are risky?
- Which future outcomes are likely?

A world model allows the agent to reason before acting.

Latent Representations

A latent representation is a compact internal representation learned from raw data.

Instead of working directly with a high-dimensional observation, o_t, the model learns a lower-dimensional representation:$z_t = e_\phi(o_t)$, where e_ϕ is an encoder.

Latent representations are useful because they can capture the information that matters while ignoring irrelevant details.

Latent Representations

- Vision: represent an image by objects, shapes, or semantic features.
- Language: represent text by meaning rather than individual words.
- RL: represent observations by control-relevant state information.

Modern AI systems often learn useful internal representations for predictions or decisions.

Latent World Models

In high-dimensional domains, the observation may be a **high-dimensional raw input**, such as an image.

Instead of predicting pixels directly,

$$o_t \to o_{t+1}$$

we learn a compact latent representation:

$$o_t \to z_t$$

In high-dimensional domains, the observation may be a **high-dimensional raw input**, such as an image.

Instead of predicting pixels directly,

$$o_t \rightarrow o_{t+1}$$

we learn a compact latent representation:

$$o_t \rightarrow z_t$$

Then the model predicts future states in latent space:

$$z_{t+1} = f_\eta(z_t, a_t)$$

Latent world models learn a compact internal representation that is easier to predict and more useful for planning.

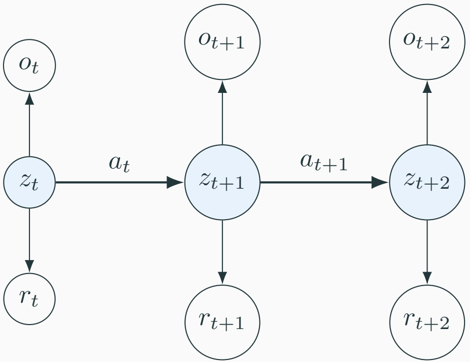
Planning in Latent Space

A latent-space model has three main components:

1. Encoder:
$$z_t = e_\phi(o_t)$$
2. Dynamics model:
$$z_{t+1} = f_\eta(z_t, a_t)$$
3. Reward model:
$$\hat{r}_t = r_\psi(z_t, a_t)$$

Latent State-Space Models

For high-dimensional observations, learn a latent state z_t.



$$p(o_t \mid z_t), \qquad p(z_{t+1} \mid z_t, a_t), \qquad p(r_t \mid z_t, a_t).$$

A latent model needs an inference model, or encoder:

$$q_\psi(z_{1:T} \mid o_{1:T}, a_{1:T}).$$

A typical objective resembles a variational lower bound:

$$\sum_{t=1}^{T} \mathbb{E}_{q_\psi}\Big[\underbrace{\log p_\phi(o_t \mid z_t)}_{\text{observation reconstruction}} + \underbrace{\log p_\phi(r_t \mid z_t, a_t)}_{\text{reward prediction}} + \underbrace{\log p_\phi(z_{t+1} \mid z_t, a_t)}_{\text{latent dynamics}}\Big] + \underbrace{\mathcal{H}(q_\psi)}_{\text{regularizes latent inference}}.$$

The model must learn both a latent representation and latent dynamics that are useful for control.

Dreamer-Style Learning

Dreamer-style agents learn a latent dynamics model and train behavior inside imagined rollouts.

Typical components:

- encoder: observation o_t to latent state z_t,
- latent dynamics: $(z_t, a_t) \to z_{t+1}$,
- reward predictor: $(z_t, a_t) \to r_{t+1}$,
- actor and critic trained using imagined trajectories.

Dreamer-Style Learning

Dreamer-style agents learn a latent dynamics model and train behavior inside imagined rollouts.

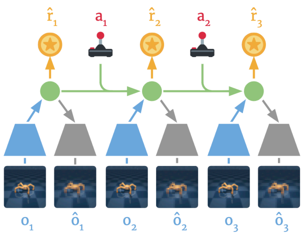
Typical components:

- encoder: observation o_t to latent state z_t,
- latent dynamics: $(z_t, a_t) \to z_{t+1}$,
- reward predictor: $(z_t, a_t) \to r_{t+1}$,
- actor and critic trained using imagined trajectories.

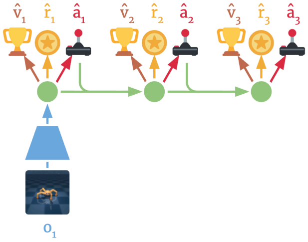
$$z_t \to z_{t+1} \to z_{t+2} \to \cdots \to z_{t+H}$$

The policy improves by practicing inside the learned world model.

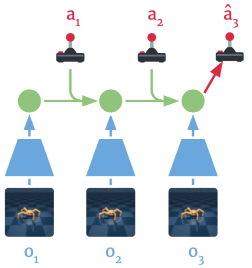
Dreamer Method



(a) Learn dynamics from experience



(b) Learn behavior in imagination



(c) Act in the environment

MuZero-Style Abstract Models

MuZero-style methods do not require a model that predicts the real next observation.

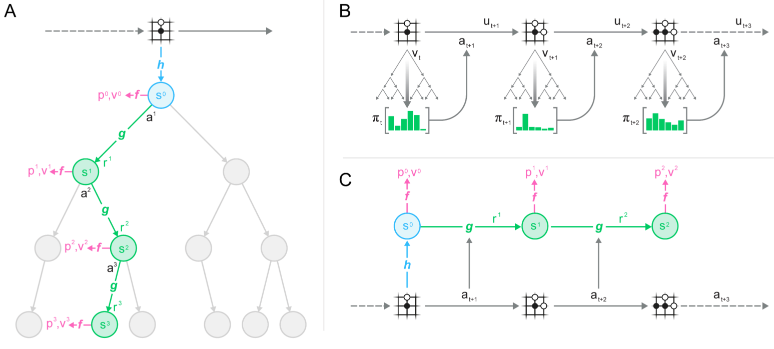
Instead, they learn three components:

- Representation model: maps observations to latent states.
- Dynamics model: predicts the next latent state.
- Prediction model: predicts policy, value, and reward.

Key idea

The model is useful for planning, even if it does not reconstruct raw observations.

MuZero Method



Priors and Foundation Models

Using Prior Knowledge

In the previous lesson, we introduced a prior model:

$$p_0(s_{t+1} \mid s_t, a_t).$$

The learned model can be trained to fit data while staying close to the prior:

$$\ell(\eta) = \sum_t \Big[-\log q_\eta(s_{t+1} \mid s_t, a_t) \\ + \beta D_{\mathrm{KL}}\left(q_\eta(\cdot \mid s_t, a_t) \,\|\, p_0(\cdot \mid s_t, a_t)\right)\Big].$$

The prior can reduce data requirements, but it can also bias the learned model.

Where Can Priors Come From?

Prior models may come from:

- physics simulators,
- analytical dynamics models,
- demonstrations,
- offline datasets,
- previously trained world models,
- vision-language-action or foundation models.

A prior model is useful when it provides structure before the agent collects enough task-specific data.

Foundation Models as Priors

Foundation models can support model-based RL by providing:

- semantic state abstractions,
- likely object affordances,
- task decompositions,
- approximate predictions of action outcomes,
- reward or goal interpretation.

Foundation Models as Priors

Foundation models can support model-based RL by providing:

- semantic state abstractions,
- likely object affordances,
- task decompositions,
- approximate predictions of action outcomes,
- reward or goal interpretation.

Examples:

- vision models provide object and scene representations,
- language models provide semantic knowledge,
- video models predict future observations,
- vision-language-action models suggest possible actions.

> The foundation model need not replace RL; it can initialize or regularize the learned world model.

Caution: Priors Can Be Wrong

Priors provide useful structure, but they should not be treated as ground truth.

A prior model may:

- be inaccurate in the current environment,
- ignore task-specific constraints,
- overstate which actions are feasible,
- introduce bias into the learned model,
- become misleading under distributional shift.

Priors are most useful when they are combined with data, uncertainty estimates, and validation in the target environment.

Distributional Shift and Model Uncertainty

Where the Simple Story Breaks

The learned model is only trained on the data distribution:

$$(s, a) \sim d^{\pi_\beta}(s, a).$$

The learned model is only trained on the data distribution:

$$(s, a) \sim d^{\pi_\beta}(s, a).$$

But after policy improvement, the new policy visits

$$(s, a) \sim d^{\pi}(s, a),$$

which may be very different.

> The policy that uses the model determines the future inputs to the model. If the policy changes too much, the model is queried out of distribution.

A basic loop is:

$$\pi_\beta \to D \to \hat{p}_\eta \to \pi_{\text{new}}.$$

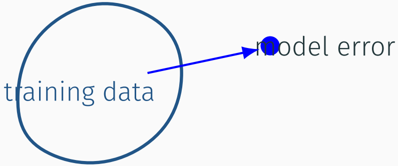
Distributional Shift in Model-Based RL

A basic loop is:

$$\pi_\beta \to D \to \hat{p}_\eta \to \pi_{\text{new}}.$$

The problem:

$$\hat{p}_\eta \text{ is accurate near } d^{\pi_\beta}, \quad \text{but } \pi_{\text{new}} \text{ may leave that region.}$$



Two Ways to Reduce Shift

1. **Do not change the policy too much.**

   $$D_{\mathrm{KL}}(\pi_{\text{new}} \| \pi_\beta) \leq \epsilon$$

   This is a trust-region style restriction.

2. **Stay where the model is confident.** Use a probabilistic model and penalize actions with high uncertainty.

The model should also tell us where it should not be trusted.

Expected Value, Optimism, and Pessimism

Suppose a model predicts a distribution over outcomes. There are different approaches that can be adopted:

- Expected value: Uses the average predicted outcome.

$$V_{\text{mean}}(s) = \mathbb{E}[V(s)]$$

- Pessimistic value: Avoids uncertain regions.

$$V_{\text{pess}}(s) = \mathbb{E}[V(s)] - \beta U(s)$$

- Optimistic value: Encourages exploration.

$$V_{\text{opt}}(s) = \mathbb{E}[V(s)] + \beta U(s)$$

From Model Uncertainty to Offline RL

So far, we assumed that when the model is uncertain, the agent may still interact with the environment.

This gives us several options:

- collect more data in uncertain regions,
- explore optimistically,
- stay close to the data distribution,
- penalize uncertain model predictions.

So far, we assumed that when the model is uncertain, the agent may still interact with the environment.

This gives us several options:

- collect more data in uncertain regions,
- explore optimistically,
- stay close to the data distribution,
- penalize uncertain model predictions.

But what if interaction with the environment is not allowed?

$$\text{fixed data} \implies \text{no new exploration} \implies \text{policy improvement must}$$

Offline RL studies the hardest version of distributional shift: the agent must improve from data, but cannot collect new data to correct its mistakes.

Offline RL as the Limit of Distributional Shift

In model-based RL, the learned model is reliable mainly near the data distribution:

$$(s, a) \sim d_{\pi_\beta}(s, a).$$

After policy improvement, the new policy may induce a different distribution:

$$(s, a) \sim d_{\pi}(s, a).$$

Offline RL as the Limit of Distributional Shift

In model-based RL, the learned model is reliable mainly near the data distribution:

$$(s, a) \sim d_{\pi_\beta}(s, a).$$

After policy improvement, the new policy may induce a different distribution:

$$(s, a) \sim d_\pi(s, a).$$

In online RL, the agent can reduce this mismatch by collecting more data.

In offline RL, the dataset is fixed

$$\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^{N}.$$

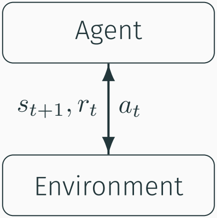
and we need to make the best decisions based on the available data.

Offline RL

Online RL vs. Offline RL

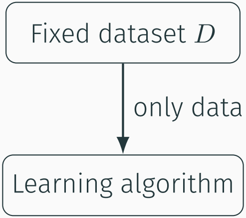
In standard online reinforcement learning, the agent repeatedly interacts with the environment.

Online RL



The agent can try new actions and collect new data.

Offline RL



The agent must learn without further environment interaction.

Offline RL learns a policy from a fixed dataset of past experience.

The dataset contains transitions collected by some behavior policy:

$$D = \{(s_i, a_i, r_i, s_i')\}_{i=1}^{N}.$$

The Offline RL Setting

The dataset contains transitions collected by some behavior policy:

$$D = \{(s_i, a_i, r_i, s_i')\}_{i=1}^{N}.$$

The goal is to learn a policy $\pi(a \mid s)$ that achieves high return when deployed.

But during training:

- the agent cannot query the environment,
- it cannot ask what would have happened under other actions,
- it only sees the actions represented in the dataset.

Why Offline RL?

Offline RL is attractive when online exploration is expensive, unsafe, or impossible.

Examples:

- robotics: real-world trial and error can damage hardware,
- healthcare: unsafe treatments cannot be explored,
- recommender systems: bad recommendations hurt users,
- energy systems: exploration can violate operational constraints,
- autonomous driving: rare dangerous events are hard to collect safely.

Offline RL tries to reuse existing data to learn better decision policies without risky online exploration.

The Central Difficulty

A supervised learning problem usually asks:

> predict well on data similar to the training data.

The Central Difficulty

A supervised learning problem usually asks:

> predict well on data similar to the training data.

Offline RL asks something harder:

> choose actions that may differ from the actions in the data.

The Central Difficulty

A supervised learning problem usually asks:

> predict well on data similar to the training data.

Offline RL asks something harder:

> choose actions that may differ from the actions in the data.

This creates a conflict:

- to improve, the learned policy may need to deviate from the data,
- but deviation makes value estimates unreliable.

Offline RL is hard because improvement requires counterfactual reasoning from limited data.

The dataset is generated by a behavior policy π_β:

$$(s, a) \sim d^\pi_\beta(s, a).$$

The learned policy induces a different distribution:

$$(s, a) \sim d^\pi(s, a).$$

If d^π places probability on state-action pairs rarely seen in D, the value function must extrapolate.

The policy is evaluated on the distribution it creates, not only on the distribution in the dataset.

Challenge 2: Support Mismatch

A severe case of distribution shift is **support mismatch**.

The learned policy may choose actions that have little or no support in the dataset:

$$\pi(a \mid s) > 0 \quad \text{but} \quad \pi_\beta(a \mid s) \approx 0.$$

Challenge 2: Support Mismatch

A severe case of distribution shift is **support mismatch**.

The learned policy may choose actions that have little or no support in the dataset:

$$\pi(a \mid s) > 0 \quad \text{but} \quad \pi_\beta(a \mid s) \approx 0.$$

Then the data provides almost no evidence for estimating:

$$Q(s, a).$$

Offline RL cannot reliably evaluate actions that were not tried in similar states.

Suppose the learned value function overestimates an unseen action:

$$\hat{Q}(s, a_{\text{unseen}}) \gg Q(s, a_{\text{unseen}}).$$

Challenge 3: Extrapolation Error

Suppose the learned value function overestimates an unseen action:

$$\hat{Q}(s, a_{\text{unseen}}) \gg Q(s, a_{\text{unseen}}).$$

A greedy policy will select this action:

$$\pi(s) = \arg\max_a \hat{Q}(s, a).$$

Then the policy exploits an error in the value estimate.

Offline RL algorithms must prevent the policy from exploiting value estimates in unsupported regions.

Challenge 4: Bootstrapping Amplifies Errors

Value-based RL uses Bellman backups:

$$\hat{Q}(s,a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s',a').$$

Challenge 4: Bootstrapping Amplifies Errors

Value-based RL uses Bellman backups:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a').$$

In offline RL, the maximization can select an action a' that was not well covered by the dataset.

Challenge 4: Bootstrapping Amplifies Errors

Value-based RL uses Bellman backups:

$$\hat{Q}(s,a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s',a').$$

In offline RL, the maximization can select an action a' that was not well covered by the dataset.

Then an error at the next state becomes part of the target.

> Bootstrapping can propagate and amplify extrapolation errors across states.

Challenge 5: Dataset Quality

Not all offline datasets are equally useful.

Important properties include:

- **coverage**: are useful states and actions represented?
- **diversity**: does the data contain multiple strategies?
- **quality**: are the trajectories mostly successful or mostly poor?
- **size**: is there enough data to estimate values reliably?
- **bias**: were some actions systematically avoided?

The best achievable policy is strongly constrained by what the dataset contains.

Imitation Is Safe but Limited

One simple baseline is behavior cloning:

$$\max_{\theta} \sum_{(s,a) \in D} \log \pi_\theta(a \mid s).$$

Imitation Is Safe but Limited

One simple baseline is behavior cloning:

$$\max_{\theta} \sum_{(s,a)\in D} \log \pi_\theta(a \mid s).$$

This keeps the learned policy close to the dataset behavior.

But it does not explicitly optimize long-term return.

Imitation Is Safe but Limited

One simple baseline is behavior cloning:

$$\max_{\theta} \sum_{(s,a)\in D} \log \pi_\theta(a \mid s).$$

This keeps the learned policy close to the dataset behavior.

But it does not explicitly optimize long-term return.

Advantage

- stable,
- simple,
- avoids unsupported actions.

Limitation

- copies mistakes,
- cannot improve much beyond the data,
- ignores reward structure.

The Offline RL Tradeoff

Offline RL must balance two goals.

Stay close to the data	Improve beyond the data
Reliable estimates, but limited improvement.	Potentially better policies, but higher risk of extrapolation error.

offline RL $=$ policy improvement under data constraints.

The central algorithmic question is how far the policy should be allowed to move away from the dataset.

Common Algorithmic Principle: Conservatism

Many offline RL methods use conservatism or pessimism.

The idea is to avoid trusting high value estimates for actions that are not well supported by the data.

Common Algorithmic Principle: Conservatism

Many offline RL methods use conservatism or pessimism.

The idea is to avoid trusting high value estimates for actions that are not well supported by the data.

Instead of learning an optimistic value function, learn a cautious one:

$$\hat{Q}(s,a) \leq Q(s,a) \quad \text{for uncertain actions.}$$

Common Algorithmic Principle: Conservatism

Many offline RL methods use **conservatism** or **pessimism**.

The idea is to avoid trusting high value estimates for actions that are not well supported by the data.

Instead of learning an optimistic value function, learn a cautious one:

$$\hat{Q}(s,a) \leq Q(s,a) \quad \text{for uncertain actions.}$$

Then policy improvement is less likely to exploit value errors.

> Pessimism protects the learned policy from unsupported actions.

Example: Conservative Q-Learning

Conservative Q-Learning penalizes high Q-values on actions not strongly supported by the data.

A simplified objective is:

$$\min_{Q} \quad \underbrace{\mathcal{L}_{\text{Bellman}}(Q)}_{\text{fit Bellman backups}} \; + \alpha \underbrace{\left(\mathbb{E}_{s\sim D,\, a\sim\pi}[Q(s,a)] - \mathbb{E}_{s,a\sim D}[Q(s,a)]\right)}_{\text{conservative penalty}}.$$

Example: Conservative Q-Learning

Conservative Q-Learning penalizes high Q-values on actions not strongly supported by the data.

A simplified objective is:

$$\min_{Q} \underbrace{\mathcal{L}_{\text{Bellman}}(Q)}_{\text{fit Bellman backups}} + \alpha \underbrace{\left(\mathbb{E}_{s\sim D,\, a\sim\pi}[Q(s,a)] - \mathbb{E}_{s,a\sim D}[Q(s,a)]\right)}_{\text{conservative penalty}}.$$

The penalty lowers values of actions proposed by the policy unless they are supported by the dataset.

CQL tries to make unsupported actions look less attractive.

Implicit Q-Learning avoids explicit maximization over unseen actions.

Instead of using

$$\max_{a'} Q(s', a'),$$

it estimates values from actions that actually appear in the dataset.

Example: Implicit Q-Learning

Implicit Q-Learning avoids explicit maximization over unseen actions.

Instead of using

$$\max_{a'} Q(s', a'),$$

it estimates values from actions that actually appear in the dataset.

The policy is then extracted by advantage-weighted regression:

$$\max_{\theta} \mathbb{E}_{(s,a)\sim D} \left[\exp\left(\frac{A(s,a)}{\lambda} \right) \log \pi_\theta(a \mid s) \right].$$

IQL improves by giving more weight to good dataset actions, without querying unsupported actions directly.

Challenge 6: Offline Policy Evaluation

Before deploying a learned policy, we want to estimate its performance offline.

This is called offline policy evaluation:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

Challenge 6: Offline Policy Evaluation

Before deploying a learned policy, we want to estimate its performance offline.

This is called **offline policy evaluation**:

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right].$$

But the same problem appears again:

- the policy may choose actions not well represented in the dataset,
- value estimates may be biased,
- importance weights can have very high variance.

Offline evaluation is itself a hard causal and statistical estimation problem.

Practical Checklist

When applying offline RL, ask:

1. What behavior policy generated the data?
2. Does the dataset cover the actions the learned policy may choose?
3. Are rewards dense and reliable?
4. Is there enough trajectory diversity?
5. Can we evaluate safely before deployment?
6. Should the method be conservative, imitation-based, or model-based?

> Offline RL is not just an algorithm choice; it is also a data-quality and evaluation problem.

Summary

Offline RL learns decision policies from fixed datasets.

The core challenges are:

- distribution shift between the dataset and the learned policy,
- support mismatch for unseen actions,
- extrapolation error in value estimates,
- bootstrapping that amplifies errors,
- dependence on dataset coverage and quality,
- difficulty of reliable offline evaluation.

The key idea is policy improvement under data constraints: improve when the data supports it, and be conservative when it does not.

Conclusion

What Did We Study in This Module?

This module focused on modern approaches that combine ideas from model-based RL, model-free RL, planning, representation learning, and offline learning.

We saw that modern RL methods often use:

- **learned models** to predict possible futures,
- **latent representations** to handle high-dimensional observations,
- **priors and foundation models** to inject structure,
- **uncertainty estimates** to know when models should not be trusted,
- **offline datasets** when new interaction is expensive, unsafe, or impossible.

> The main theme is not just learning from experience, but deciding how far we can safely generalize beyond the experience we have.

The Recurring Challenge: Leaving the Data

Across the module, the same problem appeared in different forms.

- In model-based RL, the policy may query the model outside the training distribution.
- In long model rollouts, small prediction errors may compound.
- In latent models, the representation may preserve prediction information but lose control-relevant information.
- With priors and foundation models, useful structure may also introduce bias.
- In offline RL, the learned policy may choose actions that are not supported by the dataset.

The Recurring Challenge: Leaving the Data

Across the module, the same problem appeared in different forms.

- In model-based RL, the policy may query the model outside the training distribution.
- In long model rollouts, small prediction errors may compound.
- In latent models, the representation may preserve prediction information but lose control-relevant information.
- With priors and foundation models, useful structure may also introduce bias.
- In offline RL, the learned policy may choose actions that are not supported by the dataset.

Modern RL is about controlled generalization: using models, priors, and data to improve decisions, while respecting where our estimates are unreliable.

References

- Sergey Levine, UC Berkeley CS 285/185 Deep Reinforcement Learning, Lecture 15: Model-Based RL.
- Sergey Levine, UC Berkeley CS 285/185 Deep Reinforcement Learning, Lecture 16: Model-Based RL Algorithms.
- Levine, Kumar, Tucker, and Fu. *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*. 2020.